Who Doesn't Know?

Weekly Mini Project 03 — Team 4 · Project Specification

Project	Who Doesn't Know?
Team leader	Taima Watfeh
Team members	Ali Al Hamwi, Ali Mikdad, Alaa Ismaiel, Nawar Hasan, Yazan Shartouh

> Everyone knows the secret — except one player. Ask smart questions, and find who is pretending.

1. Project Overview

Who Doesn't Know? is an online social deduction game played by a group of friends, each on their own phone or browser.

When the game starts, every player receives the same secret tip (for example "Airport") — except one randomly chosen player, the hidden player, who receives nothing and must pretend to know. Players take turns asking each other questions about the secret. The informed players try to expose the hidden player without revealing the secret too clearly; the hidden player tries to blend in. Every two rounds, the group decides whether to keep asking or to vote. If the group votes out the hidden player, the informed players win. If they accuse the wrong player, the hidden player wins.

The backend is the **single source of truth** for the whole game: the room, the players, the turns, the rounds, the decisions, the votes, the secret and the result.

2. Team

The team leader and members are listed in the table at the top of this document.

Team Responsibility

- The leader coordinates the internal work split and makes sure the team reaches a working integrated result.
- Frontend/backend ownership is not assigned by the instructor.
- The team should decide how responsibilities are distributed.
- Every member must make a meaningful contribution.
- Every member should understand the project at a high level.
- Every member must participate in the final English presentation.

3. Project Goal

Build a web application where players can:

- create a game room and share its room code;
- join the room from their own device with the room code and a display name;
- start the game once enough players have joined;
- see their own role privately: the secret tip, or a clear "you don't know" state;
- take question-and-answer turns controlled by the game;
- decide every two rounds whether to continue asking or to vote;
- vote for the player they suspect, and see the final result and reveal.

4. Technology Constraints

Area	Requirement
Frontend	React + TypeScript
Backend	NestJS
Database	MongoDB, through Mongoose
API documentation	Swagger / OpenAPI

- All game data is stored in MongoDB and must survive a backend restart.
- The game works over normal HTTP requests. Screens update after the player's own actions and by refreshing the game state regularly (polling is acceptable). Real-time WebSockets are **not** required.
- How you organize your React app and your NestJS backend is your team's decision.
- Do **not** add user accounts, login or passwords, another database, external AI services, cloud services or paid APIs.

5. Players, Room Codes and Player Ids

There is **no authentication system** in this project.

- When a player creates or joins a game, the backend creates a **temporary player id** — a random, hard-to-guess value (for example `p_a82f31`). It is returned **only** to that player.
- The player's browser keeps this id and sends it with every action, so the game knows **which player** is acting. Refreshing the page must not lose the player's place in the game.
- Every player also has a **public player reference** — for example the player's normal numeric application-level id, or another explicit public reference your team defines. This is the safe value used to show players and to **choose another player** as a question target or a vote target.
- The two identifiers have different jobs:
  - the **temporary player id** is private and is supplied only by that player's own browser when that player acts;
  - the **public player reference** identifies *other* players and may appear in the public game state.
- The public game state must **never** contain other players' temporary player ids.

> **Important:** a temporary player id is **not** authentication. It only tells the game which player a request is for. Anyone who obtained another player's id could act as that player. This is an accepted limitation of this project; real authentication comes later in the course. Do not describe it as secure.

Example:

Value	Example	Who sees it
Room code	`K7MD2`	Everyone — it is shared to invite players
Display name	Ali	Everyone in the game
Public player reference	`3`	Everyone in the game — used to pick Ali as a question or vote target
Temporary player id	`p_a82f31`	Only Ali's own browser — sent when Ali acts

6. User Scenario

Taima creates a game and becomes the host. The room code **K7MD2** appears in large letters. Ali, Nawar, Alaa and Yazan join from their phones. When five players are in the lobby, Taima starts the game.

Four phones show **"The secret: Airport"**. Nawar's phone shows **"You don't know the secret. Blend in!"**

It is Ali's turn. He chooses Alaa and asks: "Would you go there on a normal weekday?" Alaa answers: "Only if I'm in a hurry." The question and answer appear in everyone's log, and the turn moves to Nawar, who must ask something convincing without knowing the secret.

After every player has asked once, round 1 is complete. After round 2, the game asks everyone: **Continue asking or vote?** Three of five choose Continue, so rounds 3 and 4 follow. After round 4, four of five choose Vote.

Everyone votes. Nawar receives the most votes — he was the hidden player. **The informed players win.** The result screen reveals the hidden player, the secret and the vote count for every player.

7. Required Features

Feature	Description
Create game	A player creates a game with their display name, becomes the host and receives a room code.
Join game	A player joins with the room code and a display name, and receives a temporary player id.
Lobby	Everyone sees the room code and the joined players; the host can start the game.
Start game	The backend chooses the hidden player, the secret tip and the turn order.
Private role view	Each player sees only their own role information.
Public game state	Everyone sees the status, round, current turn, players and the question/answer log.
Question turns	The current player asks a chosen player; the target answers.
Round decision	Every two completed rounds, players choose Continue or Vote.
Suspect voting	Each player votes for the player they suspect.
Result and reveal	The winner, the hidden player, the secret and the vote counts.
Secret tips	A bank of secret tips, managed internally.

8. Business Rules

Game states

1. A game is always in exactly one of these states: **lobby**, **questioning**, **round_decision**, **voting**, **finished**.
2. Every action is only accepted in the state where it makes sense. Any action in the wrong state is rejected.

Joining and starting

1. A room code is 5 characters (capital letters and digits) and unique among games that are not finished.
2. Players can only join a game in the **lobby** state. Joining after the game has started is rejected.
3. Display names are required and unique within a game, ignoring upper/lower case and surrounding spaces.
4. The number of players is **not fixed**. At least **4 players** are required to start. Your team may set a practical maximum of at least 10, but no rule or calculation may depend on a specific number of players.
5. Only the host can start the game. A game that has already started cannot be started again.

When the game starts

1. The backend randomly selects **one** player as the hidden player. Every player has an equal chance.
2. The backend randomly selects **one** secret tip from the active tips.
3. Every player except the hidden player receives the same secret tip. The hidden player receives no tip and a clear "you don't know" state.
4. The backend fixes the turn order (for example in random order). The order stays the same for the whole game.

5. The game starts in **questioning** at round number **1**.

Information hiding

1. The public game state must **never** include the secret tip, the hidden player, or any field from which they could be worked out — while the game is not finished.
2. The private role view returns information only for the player whose temporary id is supplied. An unknown player id, or one that belongs to another game, is rejected.
3. Individual round decisions and suspect votes are not shown until that decision or vote is resolved. While it is open, only the number of players who have submitted is shown.
4. When the game is **finished**, the public state reveals the hidden player, the secret tip and the vote results.

Question turns

1. Only the **current player** can ask a question. Asking when it is not your turn is rejected.
2. A player cannot ask themselves. The target must be another player in the same game.
3. The question cannot be empty. After a question is asked, only the **target** can answer it, and the answer cannot be empty.
4. While a question is waiting for its answer, no new question can be asked.
5. Every question and answer is recorded with the round number, the asker and the target, and is visible to all players.
6. After the answer is recorded, the turn moves to the next player in the turn order.

Rounds

1. A round is complete when every player has completed one asking turn in that round.
2. The game tracks the current **round number**. There is **no** total number of rounds — the game can continue for as long as the players want.

Round decision (Continue or Vote)

1. At the end of every **two** completed rounds (after rounds 2, 4, 6, …), the game enters **round_decision**.
2. Each player chooses **Continue asking** or **Vote**, once per decision. A second choice in the same decision is rejected.
3. The decision is resolved when every player has chosen:
   - a **strict majority** chose Vote → the game enters **voting**;
   - otherwise — including a **tie** → the game continues: **questioning** resumes in the next round, and the round number keeps increasing.
4. Round decisions are a separate concept from suspect votes and must be stored separately.

Suspect voting

1. In **voting**, each player votes once for the player they suspect. Voting for yourself is not allowed. A second vote in the same voting phase is rejected.
2. The vote is resolved when every player has voted. The player with the **highest** number of votes is the accused.
3. If the accused **is** the hidden player → **the informed players win**.
4. If the accused is **not** the hidden player → **the hidden player wins**, because they fooled the group.
5. If two or more players **tie** for the highest number of votes → nobody wins yet. The tied result is recorded, questioning resumes in the next round, and the next round decision happens after two more completed rounds.
6. When a winner is decided, the game becomes **finished** and accepts no more actions.
7. There is no elimination: all players stay in the game until it is finished.

9. Required Screens / Views

Screen	Must show / allow
Home	Create a game (display name) or join a game (room code + display name).
Lobby	The room code in large letters; joined players; how many more are needed to start; a start action for the host only.
Game	The player's own role card (secret tip or "you don't know"); round number; whose turn it is; an ask form on your turn; an answer form when you are asked; the question/answer log.
Round Decision	Continue or Vote choice; how many players have chosen so far; the outcome once resolved.
Voting	A vote for one other player; how many players have voted; the outcome of a tie.
Result	The winner, the hidden player, the secret tip and the number of votes each player received.

All screens must be reachable in the natural flow of the game, and every player's screen must catch up with the current game state without manually re-joining.

10. Required User Flows

Flow A — Create and join

1. A player creates a game and sees the room code.
2. Other players join with the room code and their names.
3. Joining with a name already used in the room is rejected.
4. Trying to start with 3 players is rejected; with 4 or more, the host starts the game.
5. A player trying to join after the start is rejected.

Flow B — Roles

1. Every informed player sees the same secret tip.
2. The hidden player sees the "you don't know" state.
3. The public game state, viewed by anyone, contains neither the secret nor the hidden player.

Flow C — A round of questions

1. The current player asks another player a question.
2. A player who is not the current player tries to ask and is rejected.
3. The current player tries to ask themselves and is rejected.
4. The target answers; the turn moves to the next player.
5. When every player has asked once, the round number increases.

Flow D — Continue or vote

1. After round 2, the game enters the decision phase.
2. With a tie between Continue and Vote, questioning continues in round 3.
3. After round 4, a strict majority chooses Vote, and voting starts.

Flow E — Voting and result

1. Every player votes for another player.
2. If the vote is tied, questioning resumes in the next round.
3. In a later vote, one player receives the most votes.
4. The result screen shows the correct winner and reveals the hidden player and the secret.

11. Required Calculations / Derived Information

Derived information	Where it appears
Players still needed before the game can start	Lobby
Current player and next player in the turn order	Game
Turns completed and remaining in the current round	Game
Round number	Game, Round Decision, Voting
Number of players who have made their round decision / voted	Round Decision, Voting
Decision outcome (strict majority or not)	Round Decision
Vote count per player, accused player or tie	Voting, Result
Winner (informed players or hidden player)	Result

12. Validation & Error Scenarios

General expectations

- Required information cannot be missing.
- Invalid actions must be rejected — the system must never silently accept an invalid operation.
- The user always receives a clear, human-readable message explaining **why** an action failed.
- A game, room code or player that does not exist is reported as "not found".
- An action that conflicts with the current game state is reported as a conflict.
- When a request cannot be completed, the screen shows a sensible message — never a blank page.

Project-specific scenarios

Situation	Expected behavior
Joining with a room code that does not exist	"Game not found".
Joining a game that has already started	Rejected as a conflict.
Display name missing, only spaces, or already used in the room	Rejected.
Starting with fewer than 4 players	Rejected; the number of players still needed is reported.
Starting when you are not the host	Rejected.
Starting a game that has already started	Rejected as a conflict.
Asking when it is not your turn	Rejected.
Asking yourself, or a player who is not in this game	Rejected.
Asking while the previous question is still unanswered	Rejected as a conflict.
Answering a question that was not asked to you	Rejected.
Empty question or empty answer	Rejected.
Choosing Continue/Vote outside the decision phase, or twice	Rejected.
Voting outside the voting phase, voting twice, or voting for yourself	Rejected.
An unknown temporary player id, or one from another game	Rejected.
Any action on a finished game	Rejected as a conflict.

13. Data & Persistence Requirements

- Games, players, questions and answers, round decisions, suspect votes and secret tips are stored in MongoDB.
- On first start with an empty database, the backend seeds at least **30 secret tips** across at least **4 categories** (for example Places, Food, Jobs, Sports).
- Records are identified and linked using the **numeric, application-level id convention used in class** (as in B4F Board). The temporary player id described in section 5 is an additional random value used only to identify the acting player.

Data Design Decisions

Your team must make, implement and be able to explain these decisions:

1. **Relationship decision.** Decide where players, the question/answer log, round decisions and suspect votes are stored: inside the game document, or in separate collections linked by the game's id. Explain how your choice keeps the public game state free of secret information.
2. **Index decision.** Players reach a game through its room code, and the game state is requested very often. Identify the queries that run most often and add the index(es) that support them. Be ready to explain your choice.

Both decisions must be written in your repository's README.

14. Backend Quality Requirements

- **Validation:** every request body and every query parameter is validated with DTOs and the validation approach taught in class. Unknown fields are rejected.
- **Meaningful errors:** failures use the correct kind of HTTP error (bad request, not found, conflict) with a clear message.
- **Logger:** log the events that matter — game created, game started (number of players), each round completed, each decision and vote outcome, and the final result. Never log the secret tip or the hidden player before the game is finished. Do **not** log every request.
- **Swagger:** every endpoint is documented with a short summary, grouped by resource, with its request body and parameters visible. The documentation must make clear which responses are public and which are player-specific.
- **Internal-only actions:** creating, editing and deactivating **secret tips** must be protected with the `InternalToolGuard` approach taught in class (documented in Swagger, including the required header). Playing the game must **not** be guarded.
- **No fake logic:** random selection, turns, round counting, decisions, votes and the winner must be decided by the backend from stored game data. The frontend must never decide them.

15. Out of Scope

The following are **not required** and must not be added to the core project: WebSockets or Socket.IO; user accounts, login, passwords, JWT or roles; Mongoose ObjectId references or `populate`; aggregation pipelines; pagination; player elimination; chat; timers per turn; another database; external AI services; deployment.

16. Minimum Deliverable (CORE)

Your project counts as complete only when **all** of the following work:

- □ A working React + TypeScript frontend and a working NestJS backend, integrated with each other.
- □ Data is stored in MongoDB and survives a backend restart; secret tips are seeded.

- □ Every required screen in section 9 exists and is reachable in the game's flow.
- □ Every business rule in section 8 is enforced.
- □ Players create and join games with room codes; temporary player ids are issued and kept by each browser.
- □ The game works with any valid number of players (at least 4), with no fixed number of players or rounds.
- □ The secret and the hidden player never appear in the public game state before the game is finished.
- □ Turns, rounds, round decisions and voting follow the rules, including both tie cases.
- □ The winner is decided correctly and the reveal is shown.
- □ Every player's screen catches up with the game state without re-joining.
- □ Secret tips are managed only through guarded internal endpoints.
- □ Validation, error messages, Logger and Swagger meet section 14.
- □ Both data design decisions are implemented and written in the README.
- □ The README explains how to install, configure (environment variables) and run both parts, and explains the temporary player id limitation.

17. Stretch Features (optional)

Only start these after every CORE item works. They never replace a missing CORE requirement.

- **Play again** — start a new game with the same players and a new room code.
- **Tip categories** — the host chooses a category of secret tips before starting.
- **Game history** — a list of finished games with their winners.
- **Tip management screen** — an internal screen that sends the internal-tool header.

18. Presentation & Demo Acceptance

Your team presents **in English**; every member speaks. Your instructor will confirm the date and time limit. Present the game, a short walkthrough, a live demo, your data design decisions and the most important business rules — especially how the secret is kept out of the public state, and why the temporary player id is not authentication.

Prepare the demo with **at least 4 players on separate devices or separate browsers**. Two tabs of the same browser may share stored data, so plan your setup.

The live demo must pass these acceptance tests:

1. Starting with 3 players is rejected; starting with 4 or more succeeds, and joining afterwards is rejected.
2. Informed players see the same secret; the hidden player sees the "you don't know" state.
3. The public game state (shown in Swagger or the browser) contains no secret tip, no hidden player and no other players' temporary ids.
4. Asking out of turn, and asking yourself, are both rejected.
5. After two completed rounds the decision phase starts; a tie continues the game into the next round.
6. A tied suspect vote resumes questioning; a later vote produces a winner.
7. The winner matches the rules for the accused player, and the reveal is correct.
8. In Swagger, creating a secret tip without the internal-tool header is rejected, and with it succeeds.
9. Restarting the backend in the middle of a game keeps the game and every player's place.

19. Evaluation Focus

Area	What we look for
Functional completeness	The required features and screens exist and work.

Area	What we look for
Correct business behavior	States, turns, rounds, decisions, votes and the winner follow the rules exactly.
Information hiding	Secret information never leaks before the game is finished.
Frontend/backend integration	Several devices stay in sync with the backend's game state.
Backend quality	Validation, meaningful errors, Logger, Swagger and correct guard placement.
Data design	Sensible, explained relationship and index decisions.
Team collaboration and Git	Shared work, visible contributions from every member.
Explaining your own work	Every member can explain the product, its rules and its design.
Presentation quality	Clear English presentation with a working multi-device live demo.